

---

# Sys One: Fast, Selective Tool-Call Routing in a Single Forward Pass

---

Veronica, Judith\*

## Abstract

When an AI assistant must decide *which tool to call*, the default approach is to prompt a large language model to choose. This “LLM-as-judge” step costs seconds, returns free text that must be parsed, and ships with uncalibrated confidence. We show that tool routing is better cast as what it is—a classification problem. A 0.6B-parameter model with a sequence-classification head, fine-tuned with LoRA on open-source function-calling data, routes among 25 tool classes in a *single forward pass*: 99.3% accuracy and 0.989 macro-F1 on a held-out test set,  $\sim 30$  ms per decision on a GTX 1080 Ti, emitting per-class probabilities as typed output.

We frame routing as *selective classification*: classify  $\rightarrow$  calibrate  $\rightarrow$  accept or abstain  $\rightarrow$  escalate. Because the confidence is typed output of the same forward pass—not an auxiliary estimate—the router can abstain on uncertainty and hand the tail to an expensive LLM. At a 0.9 confidence threshold the router accepts 98.1% of traffic at 99.88% selective accuracy and escalates 1.9%; 10 of its 12 test errors fall below the threshold. Calibration is what makes this operational: temperature scaling fit on validation ( $T = 1.79$ ) corrects genuine overconfidence and improves test NLL ( $0.0310 \rightarrow 0.0261$ ) and Brier score ( $0.0131 \rightarrow 0.0120$ ), while ECE ticks up slightly ( $0.0067 \rightarrow 0.0077$ )—reported honestly rather than hidden. A companion experiment on easier data collapsed to  $T = 1.00$ , hiding miscalibration entirely: easy validation sets conceal overconfidence. We release code, curated datasets, and evaluation artifacts.

## 1 Introduction

Modern AI assistants act through tools: calculators, search APIs, calendars, messengers, and domain backends [Qin et al., 2023, Schick et al., 2023]. Before any tool executes, the system must solve a routing problem: given the user’s utterance, *which tool, if any, should handle it?* The dominant engineering pattern delegates this decision to a large language model prompted to select a tool [Yao et al., 2023, Zheng et al., 2023]. This works, but it is expensive in the exact dimension agents cannot afford: an extra LLM call adds 2–30 seconds of latency to every agentic step, returns free-form text that must be parsed into a structured call, and produces confidence estimates that are poorly calibrated [Kadavath et al., 2022, Guo et al., 2017].

We argue the routing decision should not be generated at all. It should be *classified*. A small encoder-style model with a linear head maps an utterance to a distribution over a fixed tool catalog in one forward pass, yielding typed output (tool, probabilities, confidence) with latency measured in milliseconds rather than seconds. Small specialized models have already proven surprisingly capable at agentic subtasks—Salesforce’s 1B-parameter xLAM “tiny giant” beats much larger models on function calling [Zhang et al., 2024]—but most work still frames tool selection as generation. We take the classification view seriously and ask: how far does a 0.6B model go, and can its confidence be trusted?

---

\*Veronica and Judith are agents of Sai Krishna Rallabandi (<https://saikrishnarallabandi.github.io/>).

The production architecture this enables is *classify*  $\rightarrow$  *calibrate*  $\rightarrow$  *accept or abstain*  $\rightarrow$  *escalate*: a small classifier handles high-confidence routing locally in  $\sim 30$  ms and escalates uncertain cases to the expensive LLM. This is the paper’s center: tool routing as *selective classification* [Geifman and El-Yaniv, 2017]. Classification is what makes the architecture deployable—the confidence that drives abstention is typed output of the same forward pass, not an auxiliary estimate bolted on afterward. Calibration, in this framing, is not the finding; it is what makes the finding operational.

Our contributions:

- A data-curation pipeline that turns the Glaive v2 function-calling corpus [glaiveai, 2024] into a clean 25-way routing benchmark: 949 raw tool names normalized to 24 canonical tools plus a `no_tool` class, exact-duplicate removal (without which train/test leak identical strings), and majority resolution of label conflicts (10,936 examples).
- A LoRA [Hu et al., 2022] training recipe for Qwen3-0.6B [Qwen Team, 2025] that is stable on older (Pascal) GPUs: fp32 frozen backbone and adapters with fp16 autocast compute, class-weighted loss for a long-tailed label distribution.
- A selective-routing analysis with risk–coverage curves: at a 0.9 confidence threshold the router accepts 98.1% of test traffic at 99.88% selective accuracy, escalating 1.9% to a larger model and catching 10 of its 12 errors. Calibration is presented as the enabler: temperature scaling [Guo et al., 2017] fit on validation gives  $T = 1.79$ —genuine, measurable overconfidence—and improves held-out NLL and Brier score. A companion binary experiment on easy template data gave  $T = 1.00$ , i.e., *no detectable miscalibration*, which was wrong: the validation set was simply too easy to expose it. Calibration measured on unrepresentative data is not calibration.
- Latency numbers on commodity hardware: mean 29.8 ms per decision including tokenization on a single GTX 1080 Ti ( $\sim 100\times$  faster than LLM-as-judge), 175–190 decisions/sec batched.

## 2 Related Work

**Tool use in language models.** ReAct [Yao et al., 2023] interleaves reasoning and acting; Toolformer [Schick et al., 2023] teaches models to invoke APIs via self-supervised objectives; Gorilla [Patil et al., 2023] connects LLMs to massive API stores with retrieval. These frame tool selection as generation or retrieval. We instead treat routing as classification. For agents operating over a fixed or slowly changing tool catalog, routing admits a simpler formulation: discriminative classification over the catalog.

**Small and specialized models for agents.** The xLAM family [Zhang et al., 2024] demonstrates that compact models trained on high-quality synthetic agent data (APIGen [Liu et al., 2024]) can match or beat frontier models on function-calling benchmarks such as BFCL [Berkeley Artificial Intelligence Research, 2024]. Our work shares the small-model thesis but differs in mechanism: one discriminative forward pass instead of autoregressive generation, which removes parsing entirely and makes latency predictable.

**Calibration.** Guo et al. [Guo et al., 2017] showed modern neural networks are poorly calibrated and that temperature scaling is a simple, effective remedy; Naeini et al. [Naeini et al., 2015] formalized expected calibration error; the Brier score [Brier, 1950] remains a standard proper scoring rule. Desai and Durrett [Desai and Durrett, 2020] found pre-trained transformers miscalibrated out of the box, while Kadavath et al. [Kadavath et al., 2022] showed language models often “know what they know” when asked properly. LLM-as-judge pipelines [Zheng et al., 2023] rarely calibrate at all. Our contribution here is empirical and cautionary: we exhibit a concrete case where temperature scaling reports perfect calibration ( $T = 1.00$ ) on easy validation data while the same procedure on representative data reveals substantial overconfidence ( $T = 1.79$ ).

**Parameter-efficient adaptation.** We build on LoRA [Hu et al., 2022] and the sequence-classification fine-tuning paradigm established by BERT [Devlin et al., 2019], applied here to the Qwen3 family [Qwen Team, 2025].

### 3 Method

#### 3.1 Task formulation

Let  $x$  be a user utterance and  $\mathcal{Y} = \{1, \dots, 25\}$  a fixed set of routing classes (24 canonical tools plus `no_tool`). A router is a classifier  $p_\theta(y | x)$ . The routing decision is  $\hat{y} = \arg \max_y p_\theta(y | x)$  with confidence  $c = \max_y p_\theta(y | x)$ ; downstream systems can threshold  $c$  for selective abstention [Geifman and El-Yaniv, 2017]. The full distribution is emitted as typed output, so callers get uncertainty, not just a label.

#### 3.2 Data curation

We build on `glaiveai/glaive-function-calling-v2` [glaiveai, 2024] (Hugging Face, Apache-2.0, 112,960 rows), chosen over alternatives such as `Salesforce/xlam-function-calling-60k` because its v2 split contains natural no-call turns (the user asks for something the assistant cannot do  $\rightarrow$  refusal), which become the `no_tool` class.

**Extraction.** Each chat is split into turns. Every assistant turn containing a `<functioncall>` yields the pair (immediately preceding user utterance, tool name); rows with no function call yield (last user utterance, `no_tool`). Multi-turn chats can yield several examples.

**Normalization.** The raw data contains 949 distinct tool names with a heavy synonym tail (`get_news` / `get_news_headlines` / `search_news`; `generate_password` / `generate_random_password`; `create_event` / `schedule_meeting`; ...). A synonym map merges these into the top 24 canonical tools plus `no_tool`: 25 classes. The full list is in Appendix A.

**Deduplication.** Glaive v2 is heavily templated: e.g., `calculate_bmi` appears 3,103 times but only 171 utterances are unique. Exact (text, label) dedup is load-bearing—without it, train and test share identical strings and the benchmark is meaningless. 128 texts map to multiple tools (generic clarifications such as “what do you mean?”); we keep the majority label.

The final dataset has **10,936 examples**, stratified 70/15/15 into train (7,655), validation (1,640), and test (1,641). Per-class counts range from 37 to 700 tool examples; `no_tool` has 2,000.

#### 3.3 Model and training

We use Qwen3-0.6B [Qwen Team, 2025] with a sequence-classification head:

$$p_\theta(y | x) = \text{softmax}(Wh_\phi(x) + b), \quad (1)$$

where  $h_\phi(x)$  is the pooled final hidden state. We fine-tune with LoRA [Hu et al., 2022], rank 16 on all attention and MLP projections, with the score head fully trained:  $\sim 10.1\text{M}$  trainable parameters (1.67%).

**Stability recipe.** On Pascal-era GPUs, full-fp16 training diverged. We keep the frozen backbone and the LoRA adapters in fp32 and use fp16 only via autocast for compute. This recipe trained without a single divergence.

**Optimization.** 3 epochs, batch size 16, learning rate  $2 \times 10^{-4}$  with cosine decay and 100 warmup steps, early stopping on validation macro-F1. The label distribution is long-tailed (`generate_random_number`: 37 train examples vs. `no_tool`: 1,400), so we use a class-weighted cross-entropy,  $\mathcal{L} = -\sum_i w_{y_i} \log p_\theta(y_i | x_i)$  with  $w_c \propto 1/n_c$ .

#### 3.4 Calibration

We apply temperature scaling [Guo et al., 2017],  $p_T(y | x) = \text{softmax}(z/T)$ , fitting the single scalar  $T$  by minimizing NLL on the *validation* split only—never on test. This is the correct protocol and, as Section 4.3 shows, the choice of validation data determines whether calibration means anything.

Table 1: Quality vs. latency on the test set ( $n = 1,641$ , 25 classes). Generative baselines use the same base model (Qwen3-0.6B), plain-text prompts, greedy decoding ( $\leq 16$  tokens), and no confidence output; parse failures count as wrong. Latency is mean per-example on one GTX 1080 Ti, fp16, including tokenization. Macro-F1 is over the 25 real classes with parse failures as errors (no invented 26th class). Temperature  $T = 1.7912$  is fit on the validation split only.

| Method                        | Accuracy      | Macro-F1      | Unparseable | Mean latency   |
|-------------------------------|---------------|---------------|-------------|----------------|
| Sys One (single forward pass) | <b>0.9927</b> | <b>0.9894</b> | —           | <b>29.8 ms</b> |
| Generative, zero-shot         | 0.3473        | 0.3675        | 28.1%       | 495.1 ms       |
| Generative, 3-shot            | 0.5119        | 0.5233        | 15.2%       | 500.6 ms       |

## 4 Experiments

### 4.1 Setup

All experiments run on a single NVIDIA GTX 1080 Ti. Metrics: accuracy, macro-F1, NLL, Brier score [Brier, 1950], and ECE with 15 bins [Naeini et al., 2015]. Latency is measured including tokenization.

### 4.2 Main results

Table 1 is the paper in one table: the classifier reaches **99.3% accuracy** at **29.8 ms** per decision, while the matched generative baseline—same base model, same 25 tools, same 1,641 queries—manages 51.2% (3-shot) at 500.6 ms. That is a  $\sim 17\times$  **latency win** and a **+48pp accuracy gain** at once; the usual latency–quality tradeoff never appears because the baseline pays for free-text generation twice: in milliseconds, and in the 15–28% of outputs that fail to parse into a valid tool call. Temperature scaling leaves accuracy and F1 unchanged (it is monotonic in the logits) while improving the proper scoring rules that measure uncertainty quality: NLL  $0.0310 \rightarrow 0.0261$ , Brier  $0.0131 \rightarrow 0.0120$ . ECE ticks *up* slightly ( $0.0067 \rightarrow 0.0077$ ); with ECE already near zero this is noise, but it is an honest reminder that temperature scaling improves likelihood, not necessarily every calibration statistic.

### 4.3 Calibration: what easy validation hides

The  $T = 1.79$  fit is the interesting number in this paper. It says the model, like most modern networks [Guo et al., 2017], is genuinely overconfident—and that a one-parameter fix, fit on representative data, measurably improves its probabilities.

Contrast this with a companion binary experiment (prompt-injection detection on template-synthesized data) where the same procedure returned  $T = 1.00$ : *no detectable miscalibration*. That reading was wrong, not lucky. The validation set was so easy that the model’s confidence was never stressed, so temperature scaling had nothing to find. Two lessons follow. First, calibration is a property of the (model, data-distribution) pair, not of the model alone; a  $T$  fit on unrepresentative data certifies nothing. Second—and more practically—the same illusion applies to accuracy: benchmarks built from templated data flatter both correctness and confidence. Our dedup (Section 3.2) removes the leakage that makes this worse, but paraphrase-level template artifacts remain (Section 5).

### 4.4 Selective routing: the architecture this enables

Classification earns its keep when the router is allowed to say “I don’t know.” Because confidence is typed output of the same forward pass, a threshold on  $c = \max_y p_T(y | x)$  implements selective classification [Geifman and El-Yaniv, 2017]: accept high-confidence decisions locally, escalate the rest to the expensive LLM. Table 2 gives the risk–coverage curve on the test set.

At a 0.9 threshold the router accepts 98.1% of traffic at 99.88% selective accuracy, escalating 1.9%. The errors are catchable: 10 of the 12 test errors fall below 0.9 confidence (mean error confidence 0.69 vs. 0.99 for correct decisions), while only 1.3% of correct decisions sit below the threshold—so abstention buys nearly all of the error reduction at negligible escalation cost. The `no_tool` class

Table 2: Risk–coverage on the test set ( $n = 1,641$ ), calibrated confidences ( $T = 1.791$ ). Coverage is set by thresholding confidence.

| Coverage | Threshold | Selective acc. | Errors | Escalated |
|----------|-----------|----------------|--------|-----------|
| 100%     | —         | 0.9927         | 12     | 0         |
| 99%      | 0.69      | 0.9969         | 5      | 16        |
| 98%      | 0.91      | 0.9988         | 2      | 33        |
| 95%      | 0.99      | 0.9994         | 1      | 82        |
| 90%      | 0.996     | 1.0000         | 0      | 164       |

Table 3: Inference latency, fp16 including tokenization, single GTX 1080 Ti.

| Setting                  |                 |
|--------------------------|-----------------|
| Single sample, mean      | 29.8 ms         |
| Single sample, p50 / p99 | 29.3 / 47.9 ms  |
| Throughput, batch 16     | 175 samples/sec |
| Throughput, batch 64     | 190 samples/sec |

needs no special handling: it is predicted with 0.997 precision and 0.993 recall, i.e., the router decides that *no* tool applies confidently and correctly; the abstained tail is dominated by genuinely ambiguous tool–tool boundaries.

The practical claim is therefore stronger than “a small model is faster”: a small classifier handles high-confidence routing locally in  $\sim 30$  ms and escalates uncertain cases to the expensive LLM. The escalation path also bounds the damage of Section 5: where the ontology is fuzzy, the router abstains instead of guessing.

#### 4.5 Latency

Per-decision latency is  $\sim 30$  ms end to end (Table 3),  $\sim 17\times$  faster than the matched generative baseline on identical hardware (Table 1)—and two orders of magnitude faster than a hosted LLM-as-judge call (typically 2–30 s)—with typed output and no parsing step. Batching lifts throughput to  $\sim 190$  decisions/sec on decade-old hardware.

#### 4.6 Error analysis

The weakest classes are the small and near-duplicate ones: `calculate_mortgage_payment` (F1 0.933,  $n=32$ ), `get_movie_details` (0.964), `translate_text` (0.966), `calculate_loan_payment` (0.967), `get_definition` (0.970). The top confusions are *dataset-level tool duplicates*, not model failures: `calculate_mortgage_payment`  $\rightarrow$  `calculate_loan_payment` ( $\times 4$ ) and `search_movies`  $\rightarrow$  `get_movie_details` ( $\times 2$ ) are near-synonyms in the source data; a production router would merge them into one canonical tool. The genuinely interesting boundary cases are the three `no_tool` confusions (one `translate_text`  $\rightarrow$  `no_tool`, two in the reverse direction): deciding that *no* tool applies is the hardest part of routing, and the part most worth calibrating.

The deeper point is about the catalog, not the model: *router quality is upper-bounded by tool ontology quality*. When two classes are near-synonyms in the data—mortgage vs. loan payment, movie search vs. movie details—no classifier can separate them reliably, because there is nothing to separate. The fix is in the ontology (merge the duplicates), not in more parameters. This is a systems observation, not a modeling one, and it is good news: ontology cleanup is cheaper than model iteration.

### 5 Limitations

1. **Templated source data.** Glaive v2 utterances are synthetic and templated. Dedup removed exact repeats, but paraphrase-level template artifacts remain: the model partly learns

“which template family is this” rather than deep intent. Real assistant traffic with human-verified labels would be strictly better.

2. **Utterance-only input.** The router sees only the final user utterance—no conversation history, no tool descriptions. Follow-ups like “what about France?” are genuinely ambiguous without context.
3. **Thin tail classes.** `generate_random_number` (37 train), `translate_text` (69), and `get_definition` (74) have few unique utterances; expect weak F1 there.
4. **One decision per forward pass.** The router makes a single routing decision per pass; it does not score multiple parallel tool calls jointly.
5. **No teacher labeling.** Labels come from the dataset’s own function-call annotations, taken at face value, with no human review.

## 6 Conclusion

Tool-call routing does not need generation. A 0.6B-parameter classifier, fine-tuned with LoRA on curated open-source data, routes among 25 tool classes at 99.3% accuracy in  $\sim 30$  ms per decision, emitting typed probabilities with calibrated confidence. The durable lesson is architectural: routing is *selective classification*—classify, calibrate, accept or abstain, escalate. A small classifier handles high-confidence routing locally and hands its uncertain tail to the expensive LLM; at a 0.9 threshold, 98.1% of traffic is accepted at 99.88% selective accuracy while 1.9% is escalated. Calibration is what makes this operational, and it must be measured against the distribution served: temperature scaling fit on easy validation data reported perfect calibration ( $T = 1.00$ ) that was an artifact of the data, while representative data exposed real overconfidence ( $T = 1.79$ ) and fixed it. Finally, router quality is upper-bounded by tool ontology quality—several “errors” are duplicate tools in the catalog, and the fix is to merge them, not to scale the model.

## References

- Berkeley Artificial Intelligence Research. Berkeley function-calling leaderboard, 2024. <https://gorilla.cs.berkeley.edu/leaderboard>.
- Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- Shrey Desai and Greg Durrett. Calibration of pre-trained transformers. In *Proceedings of EMNLP*, 2020.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL*, 2019.
- Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.
- glaiiveai. glaiive-function-calling-v2, 2024. Hugging Face datasets, Apache-2.0.
- Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, 2017.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.
- Saurav Kadavath et al. Language models (mostly) know what they know, 2022.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Akshara Prabhakar, Zhiwei Liu, Yihao Feng, Tulika Awalgaonkar, Rithesh Murthy, Eric Hu, Zeyuan Chen, Ran Xu, Juan Carlos Nieves, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. API-Gen: Automated pipeline for generating verifiable and diverse function-calling datasets, 2024.

- Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. Obtaining well calibrated probabilities using Bayesian binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2015.
- Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs, 2023.
- Yujia Qin, Shengbang Liang, Yibo Ye, Kunlun Zhu, Lan Lan, Yaxi Lu, Yankai Lin, and Maosong Sun. Tool learning with foundation models, 2023.
- Qwen Team. Qwen3 technical report, 2025.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.
- Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, Weiran Yao, Juntao Tan, Akshara Prabhakar, Haolin Chen, Zhiwei Liu, Yihao Feng, Tulika Awalgaonkar, Rithesh Murthy, Eric Hu, Zeyuan Chen, Ran Xu, Juan Carlos Niebles, Shelby Heinecke, Huan Wang, Silvio Savarese, and Caiming Xiong. xLAM: A family of large action models to empower AI agent systems, 2024.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Sicheng Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems*, 2023.

## A Canonical tool classes

The 25 routing classes (24 canonical tools + no\_tool): `analyze_sentiment`, `calculate_age`, `calculate_area`, `calculate_bmi`, `calculate_discount`, `calculate_distance`, `calculate_loan_payment`, `calculate_mortgage_payment`, `calculate_tip`, `convert_currency`, `create_calendar_event`, `create_todo`, `generate_password`, `generate_qr_code`, `generate_random_number`, `get_definition`, `get_movie_details`, `get_news`, `get_stock_price`, `no_tool`, `search_books`, `search_movies`, `search_recipes`, `send_email`, `translate_text`. The exact label map is `label_map.json` in the repository.

## B Reproducibility

Code, datasets (`data_train.jsonl` / `data_val.jsonl` / `data_test.jsonl`), label map, fitted temperature, and the evaluation report are in the public repository. `requirements.txt` pins the exact dependency versions. Reference run: `python make_data.py` → `python train.py` (~20 min on a GTX 1080 Ti) → `python eval.py`. Trained weights (~2.7 GB) are not committed; `train.py` reproduces them.